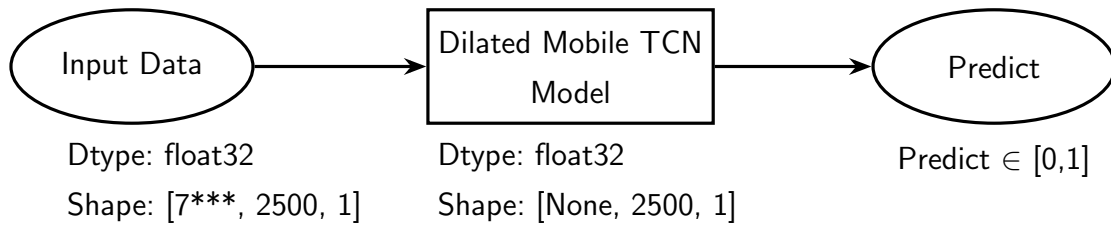


Workflow for deploying the Dilated Mobile TCN model onto MCXN947 board.

I. Model Keras float32

Với model Dilated Mobile TCN ta có quy trình sau:



Chạy thử model trên PC: 1 batch là (2500, 1) fs = 250Hz tương đương 10s

Input Data: Tập data được chọn để test là [08405, 04746, 07910, 05091] thuộc MIT-BIH Atrial Fibrillation Database, đã được tiền xử lý để loại bỏ các nhiễu và chuẩn hóa về khoảng biên độ [-3, 3].

```
--- TEST SET PERFORMANCE REPORT ---
Decision threshold: 0.5000
Total inference time: 42.7350 seconds
Average time per sample: 5.8127 ms
```

	precision	recall	f1-score	support
Normal (0)	0.9995	0.9995	0.9995	2042
AFib (1)	0.9998	0.9998	0.9998	5310
accuracy			0.9997	7352
macro avg	0.9997	0.9997	0.9997	7352
weighted avg	0.9997	0.9997	0.9997	7352

```
--- VITAL METRICS ---
Sensitivity (Recall AFib): 0.9998
Specificity (Recall Normal): 0.9995
AUC Score: 1.0000
```

Hình 1: 08405

```
--- TEST SET PERFORMANCE REPORT ---
Decision threshold: 0.5000
Total inference time: 40.6569 seconds
Average time per sample: 5.6079 ms
```

	precision	recall	f1-score	support
Normal (0)	0.9990	0.9594	0.9788	6084
AFib (1)	0.8244	0.9949	0.9017	1166
accuracy			0.9651	7250
macro avg	0.9117	0.9771	0.9402	7250
weighted avg	0.9709	0.9651	0.9664	7250

```
--- VITAL METRICS ---
Sensitivity (Recall AFib): 0.9949
Specificity (Recall Normal): 0.9594
AUC Score: 0.9989
```

Hình 3: 07910

```
--- TEST SET PERFORMANCE REPORT ---
Decision threshold: 0.5000
Total inference time: 42.1098 seconds
Average time per sample: 5.7339 ms
```

	precision	recall	f1-score	support
Normal (0)	0.9939	0.9997	0.9968	3442
AFib (1)	0.9997	0.9946	0.9972	3902
accuracy			0.9970	7344
macro avg	0.9968	0.9972	0.9970	7344
weighted avg	0.9970	0.9970	0.9970	7344

```
--- VITAL METRICS ---
Sensitivity (Recall AFib): 0.9946
Specificity (Recall Normal): 0.9997
AUC Score: 0.9999
```

Hình 2: 04746

```
--- TEST SET PERFORMANCE REPORT ---
Decision threshold: 0.5000
Total inference time: 39.8056 seconds
Average time per sample: 5.4261 ms
```

	precision	recall	f1-score	support
Normal (0)	0.9995	0.9988	0.9991	7328
AFib (1)	0.3077	0.5000	0.3810	8
accuracy			0.9982	7336
macro avg	0.6536	0.7494	0.6900	7336
weighted avg	0.9987	0.9982	0.9984	7336

```
--- VITAL METRICS ---
Sensitivity (Recall AFib): 0.5000
Specificity (Recall Normal): 0.9988
AUC Score: 0.9922
```

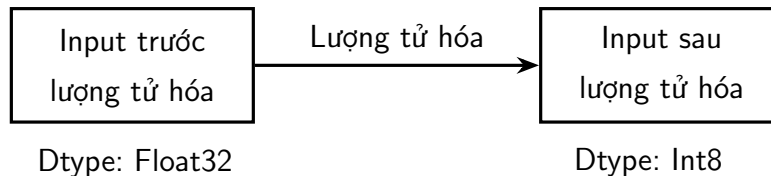
Hình 4: 05091

II. Quantization

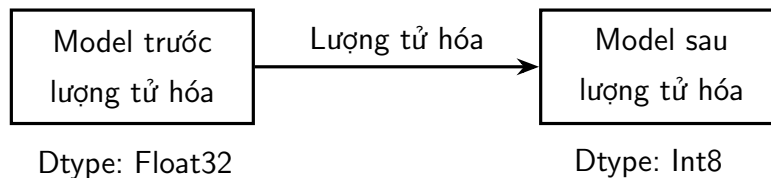
Việc lượng tử hóa (Quantization) mô hình từ kiểu dữ liệu số thực 32-bit (float32) sang số nguyên 8-bit (int8) là bước bắt buộc vì:

- **Khả năng hỗ trợ của bộ tăng tốc phần cứng (NPU).** Lõi eIQ Neutron NPU trên board MCX N947 được thiết kế với kiến trúc chuyên biệt chứa các khối tính toán nhân-cộng (MAC blocks) chỉ tối ưu hóa cho phép toán số nguyên 8-bit (INT8). Các bộ tăng tốc phần cứng AI chuyên dụng này chỉ hỗ trợ chạy các mô hình đã được lượng tử hóa.
- **Khắc phục giới hạn khắt khe về bộ nhớ.** Mỗi tham số dạng float32 tiêu tốn 4 byte bộ nhớ, trong khi dạng int8 chỉ tốn 1 byte. Việc lượng tử hóa giúp giảm kích thước của toàn bộ mô hình xuống 4 lần.
- **Tăng tốc độ suy luận.** Lõi NPU xử lý các phép tính số nguyên int8 nhanh và hiệu quả hơn rất nhiều so với tính toán số thực. Bằng cách sử dụng dữ liệu int8, lõi NPU trên MCX N947 có thể đạt tốc độ xử lý lên tới 4.8 tỷ phép tính mỗi giây (4.8 GOPS) ở xung nhịp 150 MHz. Điều này giúp tốc độ suy luận (inference) tăng lên đáng kể so với việc bắt CPU tự tính toán dữ liệu float32.
- **Tiết kiệm điện năng tiêu thụ.** Việc tính toán bằng số nguyên 8-bit nhẹ nhàng hơn, kết hợp với việc bộ nhớ nhỏ gọn giúp giảm bớt lượng dữ liệu cần truyền tải (memory I/O), từ đó giúp hệ thống mang lại hiệu suất cao nhất nhưng với mức tiêu thụ điện năng thấp nhất.

1. Quantize input data

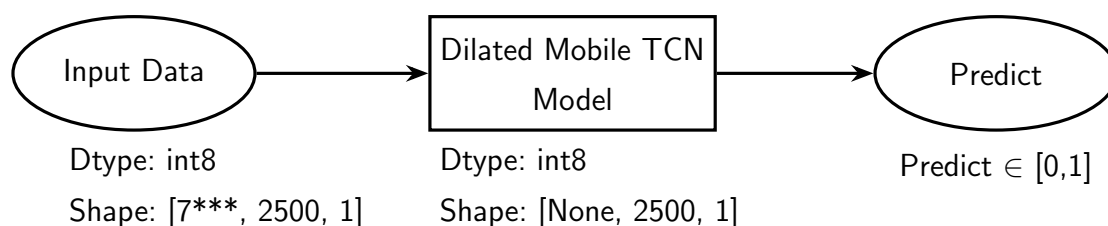


2. Quantize model



III. Model tfLite int8

Với model Dilated Mobile TCN sau khi đã lượng tử hóa ta có quy trình sau:



Trước khi convert model sang NPU để đưa xuống board ta phải kiểm tra lại xem model đã được lượng tử đúng hay chưa bằng cách chạy inference lại trên PC để xem evaluation.

```

--- THÔNG TIN MODEL ---
Input shape: [ 1 2500 1]
Input dtype: <class 'numpy.int8'>
Input quantization: (0.0235294122248888, -1)
Output shape: [1 1]
Output dtype: <class 'numpy.int8'>
Output quantization: (0.00390625, -128)
-----
Shape X: (7352, 2500, 1)
Dtype X: float32
Shape y: (7352,)
Dtype y: int32

--- TEST SET PERFORMANCE REPORT ---
Decision threshold: 0.5000
Total inference time: 10.7937 seconds
Average time per sample: 1.4681 ms

      precision    recall  f1-score   support

 Normal (0)         0.9995    0.9995    0.9995        2042
  AFib (1)          0.9998    0.9998    0.9998        5310

 accuracy                   0.9997        7352
 macro avg              0.9997    0.9997    0.9997        7352
weighted avg              0.9997    0.9997    0.9997        7352

--- VITAL METRICS ---
Sensitivity (Recall AFib): 0.9998
Specificity (Recall Normal): 0.9995
AUC Score: 1.0000
  
```

Hình 5: 08405

```

--- THÔNG TIN MODEL ---
Input shape: [ 1 2500 1]
Input dtype: <class 'numpy.int8'>
Input quantization: (0.0235294122248888, -1)
Output shape: [1 1]
Output dtype: <class 'numpy.int8'>
Output quantization: (0.00390625, -128)
-----
Shape X: (7250, 2500, 1)
Dtype X: float32
Shape y: (7250,)
Dtype y: int32

--- TEST SET PERFORMANCE REPORT ---
Decision threshold: 0.5000
Total inference time: 10.5511 seconds
Average time per sample: 1.4553 ms

      precision    recall  f1-score   support

 Normal (0)         0.9991    0.9361    0.9666        6084
  AFib (1)          0.7490    0.9957    0.8549        1166

 accuracy                   0.9457        7250
 macro avg              0.8741    0.9659    0.9107        7250
weighted avg              0.9589    0.9457    0.9486        7250

--- VITAL METRICS ---
Sensitivity (Recall AFib): 0.9957
Specificity (Recall Normal): 0.9361
AUC Score: 0.9986
  
```

Hình 6: 07910

```

--- THÔNG TIN MODEL ---
Input shape: [ 1 2500 1]
Input dtype: <class 'numpy.int8'>
Input quantization: (0.0235294122248888, -1)
Output shape: [1 1]
Output dtype: <class 'numpy.int8'>
Output quantization: (0.00390625, -128)
-----
Shape X: (7344, 2500, 1)
Dtype X: float32
Shape y: (7344,)
Dtype y: int32

--- TEST SET PERFORMANCE REPORT ---
Decision threshold: 0.5000
Total inference time: 11.0317 seconds
Average time per sample: 1.5021 ms

      precision    recall  f1-score   support

 Normal (0)         0.9942    1.0000    0.9971    3442
  AFib (1)          1.0000    0.9949    0.9974    3902

   accuracy                    0.9973    7344
  macro avg         0.9971    0.9974    0.9973    7344
weighted avg         0.9973    0.9973    0.9973    7344

--- VITAL METRICS ---
Sensitivity (Recall AFib): 0.9949
Specificity (Recall Normal): 1.0000
AUC Score: 0.9999

```

Hình 7: 04746

```

--- THÔNG TIN MODEL ---
Input shape: [ 1 2500 1]
Input dtype: <class 'numpy.int8'>
Input quantization: (0.0235294122248888, -1)
Output shape: [1 1]
Output dtype: <class 'numpy.int8'>
Output quantization: (0.00390625, -128)
-----
Shape X: (7336, 2500, 1)
Dtype X: float32
Shape y: (7336,)
Dtype y: int32

--- TEST SET PERFORMANCE REPORT ---
Decision threshold: 0.5000
Total inference time: 10.5585 seconds
Average time per sample: 1.4393 ms

      precision    recall  f1-score   support

 Normal (0)         0.9995    0.9984    0.9989    7328
  AFib (1)          0.2500    0.5000    0.3333     8

   accuracy                    0.9978    7336
  macro avg         0.6247    0.7492    0.6661    7336
weighted avg         0.9986    0.9978    0.9982    7336

--- VITAL METRICS ---
Sensitivity (Recall AFib): 0.5000
Specificity (Recall Normal): 0.9984
AUC Score: 0.9929

```

Hình 8: 05091

Bảng 1: So sánh model trước và sau lượng tử.

Record	float32				int8			
	08405	04746	07910	05091	08405	04746	07910	05091
Sensitivity (Recall AFib)	0.9998	0.9946	0.9949	0.5000	0.9998	0.9949	0.9957	0.5000
Specificity (Recall Normal)	0.9995	0.9594	0.9997	0.9988	0.9995	1.000	0.9361	0.9948
AUC Score	1.0000	0.9989	0.9999	0.9922	1.000	0.9986	0.9999	0.9929

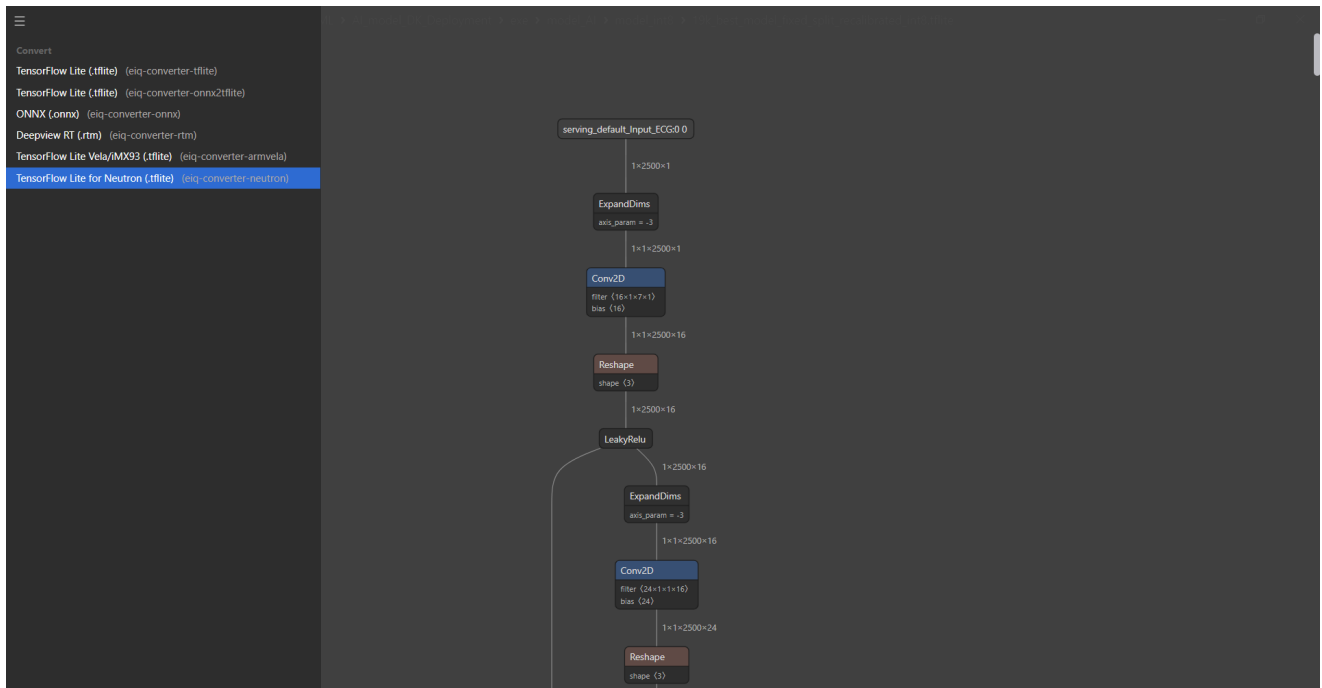
Nhận xét: Không làm suy giảm đáng kể hiệu năng. Chứng tỏ lượng tử hóa được áp dụng hiệu quả.

- AUC Score: Gần như được giữ nguyên ở mức cực cao (gần sát 1.0000).
- Ở hầu hết các bản ghi (08405, 07910, 05091), các chỉ số Sensitivity và Specificity thay đổi rất ít.

Kết luận: Lượng tử hóa thành công. Mô hình đạt được sự cân bằng giữa việc nén nhẹ tài nguyên và duy trì độ chính xác gần như tương đương với bản gốc.

IV. Nạp model xuống board mcxn947

1. Convert file.tflite cho NPU



2. Dùng sample `frdmmcxn947_tflm_cifar10_cm33_core0` trong SDK để deploy model.
3. Cập nhật lại file `model_data.h` | `model_cifarnet_ops_npu.cpp` | `labels.h` trong MCUXpresso IDE.
4. Vì `frdmmcxn947_tflm_cifar10_cm33_core0` là bài toán phân loại đa lớp (Multi-class Classification) mà bài toán của model này là phân loại nhị phân (Binary Classification) nên ta cần hiệu chỉnh lại code để có thể đọc được `Predict`.

5. Điều chỉnh lại file output_postproc.h

```
/*
 * Copyright 2020-2022 NXP
 * All rights reserved.
 *
 * SPDX-License-Identifier: BSD-3-Clause
 */

#include "fsl_debug_console.h"
#include "output_postproc.h"
#include "demo_config.h"
#include "labels.h"
#ifdef EIQ_GUI_PRINTF
#include "chgui.h"
#endif

float MODEL_GetSigmoidScore(const uint8_t* data, tensor_type_t type)
{
    float value = 0.0f;

    switch (type)
    {
        case kTensorType_FLOAT32:
        {
            const float* predictions = reinterpret_cast<const float*>(data);
            value = predictions[0];
            break;
        }

        case kTensorType_UINT8:
        {
            const uint8_t* predictions = data;
            value = predictions[0] / 255.0f;
            break;
        }

        case kTensorType_INT8:
        {
            const int8_t* predictions = reinterpret_cast<const int8_t*>(data);
            value = ((int)predictions[0] + 128) / 255.0f;
        }
    }
}
```

```

        break;
    }
}

return value;
}

status_t MODEL_ProcessOutput(const uint8_t* data, const tensor_dims_t* dims,
                             tensor_type_t type, int inferenceTime)
{
    const float threshold = (float)DETECTION_TRESHOLD / 100.0f;

    int outputCount = dims->data[dims->size - 1];
    if (outputCount != 1)
    {
        PRINTF("Error: sigmoid model expected 1 output, got %d\r\n",
               , outputCount);
        return kStatus_Fail;
    }

    float confidence = MODEL_GetSigmoidScore(data, type);
    const char* label = (confidence >= threshold) ? labels[1] : labels[0];
    int score = (int)(confidence * 100.0f);

    if (type == kTensorType_INT8)
    {
        const int8_t* out = reinterpret_cast<const int8_t*>(data);
        PRINTF("    Raw output int8: %d\r\n", out[0]);
    }

    PRINTF("-----\r\n");
    PRINTF("    Inference time: %d us\r\n", inferenceTime);
    PRINTF("    Sigmoid score : %d%%\r\n", score);
    PRINTF("    Detected      : %s\r\n", label);
    PRINTF("-----\r\n");
#ifdef EIQ_GUI_PRINTF
    GUI_PrintfToBuffer(GUI_X_POS, GUI_Y_POS, "Detected: %.20s (%d%%)"
                      , label, score);
#endif
    return kStatus_Success;
}

```

6. Điều chỉnh lại file main.cpp

```
#include "board_init.h"
#include "demo_config.h"
#include "demo_info.h"
#include "fsl_debug_console.h"
#include "model.h"
#include "output_postproc.h"
#include "timer.h"
#include "sample_idx_0.h"

int main(void)
{
    BOARD_Init();
    TIMER_Init();

    DEMO_PrintInfo();

    if (MODEL_Init() != kStatus_Success)
    {
        PRINTF("Failed initializing model" EOL);
        for (;;) {}
    }

    tensor_dims_t inputDims;
    tensor_type_t inputType;
    uint8_t* inputData = MODEL_GetInputTensorData(&inputDims, &inputType);

    tensor_dims_t outputDims;
    tensor_type_t outputType;
    uint8_t* outputData = MODEL_GetOutputTensorData(&outputDims, &outputType);

    int8_t* modelInput = reinterpret_cast<int8_t*>(inputData);

    for (int i = 0; i < SAMPLE_IDX_0_LEN; i++)
    {
        modelInput[i] = sample_idx_0[i];
    }

    auto startTime = TIMER_GetTimeInUS();
    MODEL_RunInference();
```



```

    auto endTime = TIMER_GetTimeInUS();

    MODEL_ProcessOutput(outputData, &outputDims
                        , outputType, endTime - startTime);

    PRINTF("Inference done. Program halted." EOL);

    for (;;) {}
}

```

Với `sample_idx_0.h` là file chứa input data có định dạng sau:

```

1 #ifndef SAMPLE_IDX_H
2 #define SAMPLE_IDX_H
3
4 #include <stdint.h>
5
6 #ifdef __cplusplus
7 extern "C" {
8 #endif
9
10 #define SAMPLE_IDX_LEN 2500
11 #define SAMPLE_IDX_SCALE 0.023529412225f
12 #define SAMPLE_IDX_ZERO_POINT -1
13
14 static const int8_t InputData[SAMPLE_IDX_LEN] = {
15     -76, -39, -7, 11, 14, 5, -9, -21, -28, -30, -28, -26, -25, -25, -26, -28,
16     -29, -30, -30, -28, -26, -24, -24, -24, -24, -24, -24, -23, -20, -17, -13, -9,
17     -6, -2, 2, 7, 14, 20, 26, 29, 31, 32, 31, 31, 31, 32, 33, 33,
18     31, 28, 24, 20, 15, 10, 6, 2, -3, -10, -17, -23, -28, -32, -35, -36,
19     -35, -32, -28, -23, -18, -15, -13, -11, -11, -10, -9, -9, -9, -10, -11, -12,
20     -12, -13, -13, -14, -14, -14, -14, -15, -16, -17, -18, -19, -20, -21, -22, -22,
21     -23, -23, -23, -22, -20, -18, -15, -12, -9, -9, -10, -12, -13, -13, -11, -8,
22     -7, -8, -9, -5, 10, 40, 84, 126, 126, 126, 126, 126, 96, 5, -73, -119,
23     -126, -102, -66, -34, -17, -17, -25, -37, -45, -48, -47, -44, -40, -37, -34, -32

```

7. Sau khi chạy chương trình sẽ trả ra kết quả sau:

```

CIFAR-10 example using a TensorFlow Lite Micro model.
Detection threshold: 50%
Model: Dilated_Mobile_TCN
Core/NPU Frequency: 150 MHz
TensorArena Addr: 0x200000c0 - 0x2004b0c0
TensorArena Size: Total 0x4b000 (307200 B);CIFAR-10 example using a TensorFlow Lite Micro model.
Detection threshold: 50%
Model: Dilated_Mobile_TCN
Core/NPU Frequency: 150 MHz
TensorArena Addr: 0x200000c0 - 0x2004b0c0
TensorArena Size: Total 0x4b000 (307200 B); Used 0x42da4 (273828 B)
Model Addr: 0x27e20 - 0x38480
Model Size: 0x10660 (67168 B)
Total Size Used: 340996 B (Model (67168 B) + TensorArena (273828 B))
-----
Inference time: 927006 us
Sigmoid score : 8%
Detected      : Normal
-----

```

V. Evaluation

Sau khi chạy toàn bộ mẫu (khoảng 7k3 batch) của record 08405 đưa ra được các đánh giá sau:

```
=====
FINAL EVALUATION REPORT (PC SIDE)
=====
Confusion Matrix:
  TN = 2041, FP = 1
  FN = 5, TP = 5254

Normal (0):
  precision = 0.9976
  recall    = 0.9995
  f1-score  = 0.9985
  support   = 2042

AFib (1):
  precision = 0.9998
  recall    = 0.9990
  f1-score  = 0.9994
  support   = 5259

Overall:
  accuracy   = 0.9992
  sensitivity = 0.9990
  specificity = 0.9995
  AUC Score  = 1.0000
  Avg infer  = 926719.33 us
=====
```

Hình 9: MCU

Bảng 2: Confusion matrix

	MCU	PC
TN	2041	2041
TP	5254	5258
FN	5	1
FP	1	1

Bảng 4: AFib (1)

	MCU	PC
precision	0.9998	0.9998
recall	0.9990	0.9998
f1 - score	0.9994	0.9998
support	5259	5259

```
--- TEST SET PERFORMANCE REPORT ---
Decision threshold: 0.5000
Total inference time: 42.5281 seconds
Average time per sample: 5.8250 ms

          precision    recall  f1-score   support

 Normal (0)         0.9995         0.9995         0.9995         2042
  AFib (1)          0.9998         0.9998         0.9998         5259

 accuracy                   0.9997         7301
 macro avg         0.9997         0.9997         0.9997         7301
weighted avg         0.9997         0.9997         0.9997         7301

--- VITAL METRICS ---
Sensitivity (Recall AFib): 0.9998
Specificity (Recall Normal): 0.9995
AUC Score: 1.0000
-----
```

Hình 10: PC

Bảng 3: Normal (0)

	MCU	PC
precision	0.9976	0.9995
recall	0.9995	0.9995
f1 - score	0.9985	0.9995
support	2042	2042

Bảng 5: Overall

	MCU	PC
accuracy	0.9992	0.9997
sensitivity	0.9990	0.9998
specificity	0.9995	0.9995
AUC score	1.0000	1.0000

Kết luận: Bước cơ bản đã deploy model xuống board mà không có sự sai biệt quá lớn.